

Desarrollo de Sistemas Móviles

Sesión 9: Seguridad Móvil - OWASP Top 10 y Firebase ML Kit

OWASP 2024 · Certificate Pinning · Android Keystore · ML Kit on-device

Al finalizar la clase, serás capaz de:

- **OWASP Mobile Top 10 2024:** identificar las 10 vulnerabilidades críticas y sus mitigaciones concretas.
- **Comunicación segura:** aplicar Network Security Config y certificate pinning con OkHttp.
- **Almacenamiento seguro:** usar Android Keystore para credenciales y claves criptográficas.
- **Configuración segura:** Firebase Rules por usuario, R8/ProGuard, no debuggable en producción.
- **Firebase ML Kit:** integrar IA on-device (OCR, QR, traducción) sin costo de API ni internet.

- El APK es público: cualquiera puede descargarlo y hacer reverse engineering.
- Los datos del usuario están en el dispositivo: credenciales, tokens, información personal.
- IBM Cost of a Data Breach Report 2024: el costo promedio de una brecha es USD 4.88 millones.
- Una vulnerabilidad en la app puede comprometer el backend completo (tokens robados, MITM).
- OWASP publica el ranking actualizado de vulnerabilidades móviles.
- <https://owasp.org/www-project-mobile-top-10/>

#	Vulnerabilidad	Ejemplo concreto
M1	Improper Credential Usage	API key hardcodeada en el código
M2	Inadequate Supply Chain Security	Dependencia con CVE conocido
M3	Insecure Authentication/Authorization	JWT sin expiración, sesión que no se invalida
M4	Insufficient Input/Output Validation	URL arbitraria en WebView, XSS
M5	Insecure Communication	HTTP sin cifrar, sin certificate pinning
M6	Inadequate Privacy Controls	PII en logs, permisos innecesarios
M7	Insufficient Binary Protections	APK sin ofuscación, sin R8
M8	Security Misconfiguration	Firebase Rules abiertas, debuggable=true en prod
M9	Insecure Data Storage	Contraseña en SharedPreferences sin cifrar
M10	Insufficient Cryptography	MD5, SHA1, clave hardcodeada

✗ Visible en el APK con cualquier decompilador

```
val API_KEY =  
    "sk-abc123secreto"
```

✓ local.properties (.gitignore) → BuildConfig

```
// local.properties (no commiteado)  
API_KEY=sk-abc123secreto  
  
// build.gradle.kts  
buildConfigField("String",  
    "API_KEY", "\"${properties[  
    "API_KEY"]}\"")  
  
val key = BuildConfig.API_KEY  
// ofuscado con R8 en release
```

Regla: nada secreto en el código fuente ni en el APK.

Usar local.properties para keys de desarrollo; en CI/CD, GitHub Secrets o variables de entorno.

M2: Supply Chain Security

Dependabot crea PRs automáticos cuando hay actualizaciones de seguridad:

```
# .github/dependabot.yml
updates:
  - package-ecosystem: "gradle"
    directory: "/"
    schedule:
      interval: "weekly"
    security-updates-only: true
```

M3: Autenticación insegura

- Access token corto (15 min) + refresh token largo (7 días).
- Firebase Auth gestiona expiración automáticamente.
- Interceptor Retrofit: renovar el token al recibir 401 antes de mostrar error al usuario.

Threat modelado (STRIDE): M2 y M3 se mapean a Tampering (dependencias comprometidas) y Spoofing (robo de identidad/tokens).

M4: Validar inputs, especialmente en WebViews

```
// ❌ Carga cualquier URL: puede cargar contenido malicioso
webView.loadUrl(intent.getStringExtra("url"))

// ✅ Validar dominio antes de cargar
fun loadSafeUrl(url: String) {
    require(url.startsWith("https://miapp.com/")) { "URL no permitida" }
    webView.loadUrl(url)
}

webView.settings.javaScriptEnabled = false // deshabilitar
```

M5: Forzar HTTPS en todas las conexiones (Network Security Config)

```
<!-- res/xml/network_security_config.xml -->
<network-security-config>
    <base-config cleartextTrafficPermitted="false"/>
</network-security-config>
```

Un ataque MITM (Man-in-the-Middle) intercepta el tráfico aunque uses HTTPS, usando un certificado falso instalado en el dispositivo.

Certificate Pinning rechaza cualquier certificado que no sea el tuyo, aunque sea “válido”.

```
val certificatePinner = CertificatePinner.Builder()
    .add("api.miapp.com", "sha256/HASH_DEL_CERTIFICADO=")
    .add("api.miapp.com", "sha256/HASH_BACKUP=") // siempre backup pin
    .build()

val client = OkHttpClient.Builder()
    .certificatePinner(certificatePinner)
    .build()
```

Siempre incluir un backup pin: si solo tienes uno y el cert se renueva, la app deja de funcionar.

M6: Nunca loguear datos sensibles

```
// ❌ Log.d("Auth",  
//   "Password: ${pwd}")  
  
// ✅ proguard-rules.pro:  
-assumenosideeffects  
  class android.util.Log  
  { *; }
```

M7: Binary Protections (R8)

```
release {  
    isMinifyEnabled  
        = true  
    isShrinkResources  
        = true  
}
```

M8: Firebase Rules correctas

```
// ❌ allow read,  
//   write: if true;  
// ✅ dueño del doc:  
match /usuarios/  
  {userId}/{d=**}  
  allow read, write:  
    if auth.uid==userId;
```

- M6: PII en logs, permisos innecesarios.
- M7: APK sin ofuscación expone lógica de negocio.
- M8: Firebase Rules abiertas o debuggable=true en producción.

Threat Modeling: STRIDE (I / T / E)

Info Disclosure → BuildConfig + R8 · Tampering → certificate pinning · Elevation of Privilege → Firebase Rules por UID

M9: Nunca guardar secretos en SharedPreferences

```
// ❌ SharedPreferences: texto plano, legible con root
sharedPreferences.edit().putString("token", accessToken).apply()

// ✅ Android Keystore: la clave nunca sale del hardware seguro
val keyGenerator = KeyGenerator.getInstance(
    KeyProperties.KEY_ALGORITHM_AES, "AndroidKeyStore")
keyGenerator.init(KeyGenParameterSpec.Builder("MiAlias",
    PURPOSE_ENCRYPT or PURPOSE_DECRYPT)
    .setBlockModes(KeyProperties.BLOCK_MODE_GCM).build())
```

M10: Criptografía correcta

- ❌ MD5, SHA-1 → rotos, no usar para seguridad.
- ✅ SHA-256 mínimo para hashes de integridad.
- ✅ AES-256-GCM para cifrado simétrico.
- ✅ Contraseñas: bcrypt / Argon2 en el servidor, nunca en el cliente.

Antes de implementar, identificar las amenazas del sistema:

Letra	Amenaza	Ejemplo en app móvil	Mitigación
S	Spoofing	Robo de JWT	Tokens de corta duración
T	Tampering	MITM en la API	Certificate pinning
R	Repudiation	Sin log de auditoría	Logs en servidor
I	Info Disclosure	API key en el APK	BuildConfig + R8
D	Denial of Service	Tumbar el servicio	Rate limiting en API Gateway
E	Elevation of Privilege	Acceder a datos de otro user	Firebase Rules por UID

Modelos de IA que corren en el dispositivo: sin internet, sin costo de API, sin latencia de red.

Feature	Descripción	Caso de uso
Text Recognition v2	OCR: extraer texto de imágenes	Escáner de documentos, recibos
Barcode Scanning	QR, EAN-13, UPC, Code128...	Control de inventario, pagos
Face Detection	Detectar rostros y expresiones	Filtros AR, verificación de identidad
Object Detection	Identificar objetos en imagen	Clasificación de productos
Language ID	Detectar idioma de un texto	Apps multidioma
Translation	17 idiomas, completamente offline	Chat sin internet
Pose Detection	Postura del cuerpo completo	Apps de fitness, fisioterapia

```
implementation("com.google.mlkit:text-recognition:16.0.1")
implementation("com.google.mlkit:text-recognition-latin:16.0.1")

fun reconocerTexto(bitmap: Bitmap, onResult: (String) -> Unit) {
    val image = InputImage.fromBitmap(bitmap, 0)
    val recognizer = TextRecognition.getClient(TextRecognizerOptions.DEFAULT_OPTIONS)

    recognizer.process(image)
        .addOnSuccessListener { result ->
            onResult(result.text) // texto completo reconocido
        }
        .addOnFailureListener { e -> /* manejar error */ }
}
```

Reconoce texto en español, inglés y otros idiomas latinos con el modelo text-recognition-latin. Devuelve bloques de texto con sus coordenadas en la imagen: útil para formularios. Corre completamente on-device.

Barcode Scanning

```
implementation("com.google.  
mlkit:barcode-scanning:17.3.0")  
  
fun escanearCodigo(bitmap: Bitmap,  
    onResult: (String) -> Unit) {  
    val image = InputImage.fromBitmap(bitmap, 0)  
    val scanner = BarcodeScanning.getClient(  
        BarcodeScannerOptions.Builder()  
            .setBarcodeFormats(  
                Barcode.FORMAT_QR_CODE,  
                Barcode.FORMAT_EAN_13  
            ).build()  
    )  
    scanner.process(image)  
        .addOnSuccessListener { codes ->  
            codes.firstOrNull()?.rawValue  
                ?.let { onResult(it) }  
        }  
}
```

SonarQube: Análisis de calidad y seguridad

```
# .github/workflows/sonar.yml  
- name: SonarQube Scan  
  run: ./gradlew sonar  
  env:  
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
```

Detecta automáticamente: hardcoded secrets, SQL injection patterns, code smells.

Vulnerabilidad	Mitigación clave
M1 Credentials	BuildConfig + local.properties + .gitignore
M2 Supply Chain	Dependabot, revisar CVEs de dependencias
M3 Auth/Authz	Access token (15 min) + refresh (7 días), Firebase Auth
M5 Communication	HTTPS siempre, Network Security Config, certificate pinning
M7 Binary	isMinifyEnabled = true, R8/ProGuard en release
M8 Misconfiguration	Firebase Rules por uid, debuggable=false en prod
M9 Data Storage	Android Keystore, nunca SharedPreferences para secretos
M10 Cryptography	AES-256-GCM, SHA-256, contraseñas con Argon2 en servidor
STRIDE	Modelar amenazas antes de implementar
ML Kit	OCR, QR, traducción on-device: sin internet, sin costo



Lab

Laboratorio, Semana 10 (14)

10 am

Gracias

